

CURSO DE REACT / AULA 04

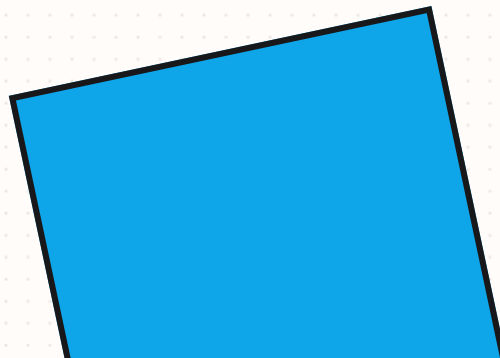
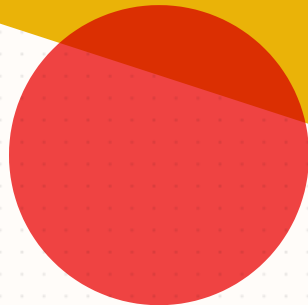
CONSUMO DE API

Fetch puro, Axios e os três estados de uma requisição.

1H30 - 2H

6 BLOCOS

CÓDIGO AO VIVO



ABERTURA

O APP ERA UMA ILHA

TO-DO

As tarefas nasciam no `useState`.

CONTADOR

O número começava do zero, sempre.

VITRINE

Os produtos eram um array no código.

Nenhum deles sabia de nada que acontece no mundo lá fora.

NOS APPS DE VERDADE

OS DADOS MORAM FORA DO APP

IFOOD

busca os restaurantes num servidor

INSTA

busca os posts do seu feed

BANCO

busca o seu saldo em tempo real

Hoje o nosso app aprende a conversar com o mundo.

01

BLOCO 1 — 18 MIN

O QUE É UMA API

Zero código aqui. Primeiro a ideia.

METÁFORA

A API É O GARÇOM

VOCÊ

O APP

Quer comida — ou
seja, dados.

GARÇOM

A API

Leva o pedido e traz
a resposta.

CARDÁPIO

ENDPOINTS

O que você pode pedir, item
por item.

COZINHA

SERVIDOR

Prepara o prato. Você não
entra lá.

Esqueça a sigla. Lembre do garçom.

O VOCABULÁRIO

REQUISIÇÃO

O pedido que o app faz: “me traz os dados do Pikachu”.

RESPOSTA

O que volta do servidor: os dados do Pikachu.

ENDPOINT

Um endereço (URL) para cada tipo de pedido.

JSON

O idioma universal em que a comida chega.

O IDIOMA DOS DADOS

JSON É SÓ UM OBJETO

Chave e valor, igualzinho a um objeto JavaScript.
Python, Java, C# — todo mundo fala JSON.

```
{  
  "nome": "Pikachu",  
  "tipo": "elétrico",  
  "altura": 4,  
  "peso": 60}
```

CURIOSIDADE

Crockford não inventou um formato novo: pegou a sintaxe de objeto do JavaScript.

Virou o idioma universal da internet — até em linguagens que não têm nada a ver com JS.

STATUS HTTP

200

Deu tudo certo, aqui está o seu prato.

404

Esse prato não existe no cardápio.

500

A cozinha pegou fogo — erro do servidor.

Existe até um status que é piada oficial: **418 — I'M A TEAPOT**. Nasceu num 1º de abril e virou padrão.

MÃO NO NAVEGADOR

DINÂMICA 1 — 8 MIN

ESPIANDO A API SEM CÓDIGO

```
https://pokeapi.co/api/v2/pokemon/pikachu
```

```
https://viacep.com.br/ws/50030230/json/
```

Sem React, sem código: a URL é o pedido, o JSON é a resposta. API é um site que responde com dados em vez de uma página bonita.

02

BLOCO 2 — 25 MIN

FETCH PURO

Já vem no navegador. Nada para instalar.

CONCEITO-CHAVE

BUSCAR DADOS NÃO É INSTANTÂNEO

O CAFÉ

Você não fica congelado olhando a máquina. Pede e faz outras coisas enquanto espera.

NO CÓDIGO

Esse “espera ficar pronto” tem nome: `await`.

Meio segundo, dois segundos — depende da internet e do servidor. Isso é assíncrono.

FETCH EM DOIS TEMPOS

```
async function buscarPokemon() {  
  const resposta = await fetch("https://pokeapi.co/api/v2/pokemon/pikachu");  
  const dados = await resposta.json();  
}
```

1. AWAIT FETCH()

Traz a resposta crua — o pacote fechado.

2. AWAIT .JSON()

Abre o pacote e converte para JSON utilizável.

Esquecer o segundo await é a pegadinha nº 1.

ATENÇÃO

O FETCH NÃO TRATA 404 NEM 500 COMO ERRO

Se o servidor respondeu alguma coisa — mesmo “não encontrado” — para o fetch a missão foi cumprida. Quem checa é você:

```
if (!resposta.ok) {  
  throw new Error("A resposta não veio OK");  
}
```

CURIOSIDADE — CONEXÃO COM A AULA 2

O fetch é a versão civilizada do XMLHttpRequest — o AJAX que fez o Gmail parecer mágica em 2004.

Buscar dados sem recarregar a página é o que permitiu as SPAs existirem.

03

BLOCO 3 — 22 MIN

OS TRÊS ESTADOS

O padrão que vocês vão repetir a vida inteira.

TRÊS MOMENTOS

ESTADO 01

CARRE- GANDO

O pedido foi feito, a resposta ainda não chegou.

ESTADO 02

ERRO

Internet caiu, servidor falhou.
Mensagem amigável na tela.

ESTADO 03

SUCESSO

Os dados chegaram. Agora sim:
mostre os dados.

CONEXÃO COM A AULA 1

FEEDBACK NÃO É ENFEITE — É RESPEITO

Tela congelada e branca sem avisar nada: o usuário acha que travou e fecha o app. Um “carregando...” separa a experiência boa da frustrante.

O PADRÃO COMPLETO

```
const [dados, setDados] = useState(null);
const [carregando, setCarregando] = useState(true);
const [erro, setErro] = useState(null);
useEffect(() => {
  async function buscar() {
    try {
      const resposta = await fetch(url);
      const json = await resposta.json();
      setDados(json);
    } catch (e) {
      setErro("Não foi possível carregar");
    } finally {
      setCarregando(false);
    }
  }
  buscar();
}, []);
```

```
if (carregando)
  return <p>Carregando ... </p>;
if (erro) return <p>{erro}</p>;
return <h1>{dados.name}</h1>;
```

3 USESTATE

Um para cada estado.

TRY/CATCH/FINALLY

O finally sempre desliga o carregando.

FOTOGRAFEM ESTE SLIDE

ALERTA DE LOOP INFINITO

O `[]` NO FIM DO USEEFFECT É SAGRADO

BUSCA



ATUALIZA STATE



REDESENHA



PARA SEMPRE

Centenas de requisições por segundo: o navegador trava e o servidor pode te bloquear. Array vazio significa buscar uma vez só.

BLOCO 4 — 20 MIN

AXIOS

A mesma busca, com uma biblioteca da comunidade.

A MESMA BUSCA COM AXIOS

```
npm install axios
```

```
import axios from "axios";  
  
const resposta = await axios.get(url);  
console.log(resposta.data);
```

Um `await` só: o Axios converte o JSON sozinho e entrega os dados prontos em `resposta.data`.

FETCH VS AXIOS

FETCH

Zero instalação — já vem no navegador.

Uma dependência a menos no projeto.

Suficiente para a maioria dos casos simples.

AXIOS

Converte o JSON automaticamente.

404 e 500 caem direto no catch.

Sintaxe curta e recursos para apps grandes.

Projeto pequeno: fetch resolve. App grande: o Axios facilita. **Saibam ler e escrever os dois.**

05

BLOCO 5 — 20 MIN

MÃO NA MASSA

APIs gratuitas, sem cadastro e sem chave.

O CARDÁPIO

VIACEP

viacep.com.br

CEP entra, endereço sai. Brasileiríssima e útil em formulário real.

POKÉAPI

pokeapi.co

Todos os dados de todos os Pokémon. Viciante e rica.

RICK & MORTY

rickandmortyapi.com

Personagens já com paginação — boa para listas grandes.

JSONPLACE

jsonplaceholder.typicode.com

API de mentirinha que aceita até enviar dados com POST.

DOG / CAT

dog.ceo • thecatapi.com

Uma foto aleatória por requisição. Efeito visual imediato.

AO VIVO

BUSCADOR DE CEP

Um input, um botão, uma requisição ao ViaCEP e os três estados tratados. A mesma funcionalidade de qualquer e-commerce de verdade.

CEP

50030-230

BUSCAR ENDEREÇO

Rua do Bom Jesus

Recife Antigo · Recife / PE

AVISO ANTECIPADO

CORS É UM MURO DO NAVEGADOR

O QUE FAZ

Decide quais sites podem chamar quais APIs.
Algumas liberam, outras bloqueiam.

SE APARECER

Não é bug seu: é a API dizendo “você não tem permissão de me chamar daqui”.

Todas as APIs do cardápio liberam. Aprofundamos depois.

ERROS COMUNS DA TURMA

2º AWAIT

Sem resposta.json(), os dados não aparecem.

ARRAY VAZIO

Loop infinito de requisições até travar.

ERRO NO FETCH

404 não cai no catch do fetch. No Axios, cai.

NOME DO CAMPO

É dados.name, não dados.nome. Leia o JSON real.

SEM CARREGANDO

A tela quebra lendo dados.name com dados null.

CORS

O erro vermelho não é culpa do seu código.

06

BLOCO 6 — 10 MIN

FECHAMENTO

Quatro aulas, uma escada.

QUATRO AULAS

01

DESIGN

O que conquista o usuário.

02

INTERFACE

Componentes, props e rotas.

03

HOOKS

Memória e reação.

04

API

Conexão com o mundo real.

Buscar, mostrar e reagir a dados reais: o kit básico de um dev front-end.

DESAFIO DA SEMANA

1 Escolha uma API do cardápio — PokéAPI ou Dog API para começar.

2 Monte uma tela que busca os dados ao carregar e mostra na tela.

3 **OBRIGATÓRIO: trate os três estados — carregando, erro e sucesso.**

4 Faça uma versão com fetch e outra com axios, para sentir a diferença.

5 Bônus: um botão “buscar outro” que traz um dado novo a cada clique.

DE VITRINE ESTÁTICA A APP CONECTADO

Bom código — e até a próxima.